Quetrex — New User Setup

Get your machine and repositories connected. One-time setup, then you're working.

Welcome. Quetrex is your team's project board at **dash.quetrex.com** plus a set of terminal commands that do the actual work. The board is where you **see** what's happening; the commands are how you **do** things. This guide gets you set up and explains how everything fits together.

Part 1 · One-time setup on your machine

You only do this **once per computer**.

1. **Accept your invite.** You'll receive an email from `noreply@quetrex.com`. Click the link, set your password, and you're in at **https://dash.quetrex.com**. You'll only see the projects you've been given access to — that's expected.

2. **Install Quetrex:** `npm install -g quetrex-base`

3. **Log in:** `/quetrex-login` — opens your browser to approve this device, then stores your access on this machine. You won't repeat it until it expires.

Part 2 · Connect each repository

For **each** repo you work in, run `/quetrex-init` once.

1. **A brand-new repo:** it asks for a project name, creates the project on the board (assigning its code, e.g. `SMA`), and links the repo.

2. **A repo already set up by a teammate** (one you've joined): it detects the existing link and simply connects you — no project is re-created. If you don't have access yet, it tells you to contact your administrator.

> **Your existing work is safe.** `/quetrex-init` is non-destructive by design. It **checks whether the repo has already been initialized and will not overwrite a previous setup.** It keeps your existing `CLAUDE.md`, custom commands, settings, and your entire git & Claude history — it only **adds** the Quetrex link and tidies up any stale references from older tooling. Nothing you've built is lost.

> **Adopting an existing repo?** `/quetrex-init` also makes sure the project's **Verification commands** are in place and offers to clean up stale commands left by an older tracker. And if it finds credentials already in your

> `.env` / `.env.*` files (database URLs, tokens, app keys), it offers — with your confirmation — to **import them into the project's secure vault**, so you don't have to re-enter them by hand. See **Part 3**.

Part 3 · Secrets & deploy keys

Projects need secrets — a `DATABASE_URL`, a Fly.io deploy token, third-party app keys. In Quetrex these live in a **secure vault**, not in your terminal or in plain files.

1. **You enter them in the browser**, never in the terminal — go to **dash.quetrex.com/keys** and add each project's deploy and runtime secrets there.
2. **They're stored encrypted** and **scoped** to the people with access to that project. Once saved, they're shown **masked** (only the last 4 characters) — even to you.
3. **Skills fetch them securely at run time.** When a command like `/deploy` runs, it pulls the secrets it needs in-memory; they're never printed or written to disk.
4. **Rotating a secret?** Just re-enter the new value at **/keys** — the next run picks it up automatically.

> Already have secrets in a repo's `.env` files? `/quetrex-init` can **import them into the vault for you** (with confirmation) when you connect the repo — see Part 2.

Part 4 · The commands

COMMAND	WHAT IT DOES
`/quetrex-login`	Authenticates your machine via your browser. One-time (until it expires). Run it again if a command says you're not logged in.
`/quetrex-init`	Connects a repository to its Quetrex project. Safe on existing repos — never overwrites your prior setup.
`/new-task`	Creates a task in the **Backlog** — title, description, assignee, and priority.
`/refine-task SMA-1`	Sharpens a vague task into a clear spec (with acceptance criteria) before building — an interactive dialog grounded in the code. Run it before `/que-task`; it doesn't start work.
`/que-task SMA-1`	Queues a task and starts the work. An agent reviews the task and the code, asks anything unclear, then builds it (see Part 5). The board moves it to **In Progress**, then **PR Ready**.

`/rework SMA-1`	For a task that needs another pass. It explains what went wrong, talks the fix through with you, notes the plan on the task, and re-runs the work.
`/task-complete SMA-1`	Marks a deployed task as **Complete** after you've confirmed it.
`/task-merge SMA-1`	Merges the task's pull request and cleans up — no leftover branches or worktrees.
`/deploy`	Deploys the project.

Part 5 · The board & task lifecycle

The board is an **observability layer** — you watch progress here; you act through the commands. A task moves through these columns:

Backlog	Created with `/new-task`, not started. Optionally sharpen it first with `/refine-task`.
Queued	Picked up with `/que-task`; about to be worked.
In Progress	An agent workflow is actively building it.
PR Ready	Work done; a pull request is open for review.
Merged	The PR has been merged with `/task-merge`.
Deployed	Released with `/deploy`.
Needs Clarity	The agent couldn't finish and left a note explaining why — pick it up with `/rework`.
Complete	Confirmed done with `/task-complete`.

Part 6 · How your work gets built

Before queuing a vague task, it's worth running `/refine-task SMA-1` first: it opens an interactive dialog — grounded in your actual code — that turns a fuzzy idea into a clear spec with **acceptance criteria**, then writes the sharpened description back to the board. It doesn't start any work; it just makes the build that follows far more accurate. When you run `/que-task`, Quetrex runs a strict, automated development pipeline so work is consistent and reviewed:

It creates an **isolated branch and worktree**, then runs a **workflow** of specialized agents: the **architect** plans, one or more **developers** implement with tests, **QA** proves everything passes, the **reviewer** checks logic and security, and a **pull request** is opened for you. It all runs in the **background**, so your terminal is free to start the next task. You watch progress on the board.

Part 7 · Watching a build with `/workflows`

When you run `/que-task`, the pipeline above runs as a **background workflow** — that's why your terminal stays free. To watch it live, type `/workflows`.

It opens a **live, auto-updating view** of every build in progress. Each one shows a progress tree:

Phases	The big steps (Architect → Developers → QA → Reviewer → PR); the current one is highlighted.
Agents	Each specialized agent under its phase, marked queued → running → done.
Status lines	Short live messages from agents as they work.

So at a glance you know **exactly where your task is** — which phase is running, what's finished, what's left — and you'd spot a stall or failure immediately. It's **read-only** (you watch; the commands drive), and you don't need to keep it open — you're notified when a build finishes — but it's your window into work in flight.

Part 8 · Roles & access

You only see and work on the projects you've been granted — everything else is invisible to you. Access is enforced on the server, so it's a hard boundary, not just hidden in the interface.

Super Admin	Full access to all projects; manages admins and all access.
Admin	Manages access for the projects they belong to.
User	Works on assigned projects; views the board for those projects.

> If you try to act on a project you don't have access to, Quetrex stops you and tells you to **contact your administrator**. Getting added to a project takes effect immediately — nothing to reinstall.

Part 9 · For administrators

From the dashboard you manage people and access — no terminal needed:

Invite someone	Click **Invite**, enter their email, choose their role, and pick which projects they get. They receive an email to set their password.
Grant / revoke access	Open **Members / Access**, select a person, and check or uncheck projects. Changes are instant — the user does nothing.

Create a project	Done from the terminal with `/quetrex-init` in the repo — it creates the project and links the code automatically.